

pyside6-webusb v0.0.5b2

vs Chrome WebUSB Compatibility Matrix

English edition • GitHub/release label: v0.0.5b2 • package version: 0.0.5.post6

Scope and interpretation

Strict technical comparison of the PySide6 / QtWebEngine implementation against the WebUSB model exposed by Chrome / Chromium. This edition updates the supplied v0.0.5a2 matrix for the v0.0.5b2 source tree and preserves its conservative compatibility terminology. The package metadata and shipped artifacts use **0.0.5.post6**; v0.0.5b2 is the informal GitHub/development label.

Chrome / Chromium is the compatibility reference. pyside6-webusb is a host-application implementation using QtWebEngine, QWebChannel, pyusb/libusb, and optional Rust acceleration. A matching method name does not automatically mean browser-level equivalence.

Status legend

Status	Meaning
Full	Behavior is intentionally implemented to closely match the Chrome/WebUSB model in the supported host environment.
Partial	The API or behavior exists, but there are documented differences, host-environment limits, or incomplete edge cases.
Host-specific	Implemented by pyside6-webusb for desktop/embedded applications and not a browser WebUSB primitive.
Different	The capability exists through a different architecture or semantics; it should not be treated as Chrome-equivalent.
Limited / unverified	Implemented or represented, but physical-hardware coverage or backend behavior is not yet sufficiently verified.
N/A	Not meaningfully comparable because the feature belongs to Chrome's browser process or another browser-only layer.

Release context

v0.0.5b2 is marked **Experimental Beta**. The v0.0.5b2 changelog records 211 Python/security-audit tests passed and 2 skipped, plus Node polyfill and TypeScript checks remaining passing. Automated tests do not replace testing against real USB devices.

1. Global architecture

Feature / Operation	Chrome / Chromium	pyside6-webusb v0.0.5b2	Assessment
Web-page API	Browser-provided WebUSB implementation	Injected JavaScript WebUSB polyfill	Partial / Different
Browser bridge	Internal browser implementation	QWebChannel + WebUSBBridge	Different
Native USB layer	Browser/OS USB stack	pyusb/libusb	Different
Device chooser	Browser-controlled chooser	Native device chooser integrated with host bridge	Partial
Permission store	Browser permission model	Origin-aware host-side permission management	Partial / Host-specific
Browser process isolation	Chromium browser architecture	QtWebEngine application architecture	Different
Native acceleration	Not a WebUSB API feature	Optional Rust acceleration	Host-specific
Environment diagnostics	Chrome internal diagnostics / DevTools	pyside6-webusb-doctor and Python report APIs	Host-specific
Virtual USB devices	Not a normal browser primitive	VirtualUsbDevice / VirtualUsbBackend for repeatable host-side testing	Host-specific / Extra
Descriptor replay	Not a normal browser primitive	VirtualUsbDevice.from_descriptor() can replay a captured descriptor without hardware	Host-specific / Extra

2. Core WebUSB surface

Feature / Operation	Chrome / Chromium	v0.0.5b2	Assessment
navigator.usb	Yes in supported secure contexts	JavaScript polyfill provides navigator.usb	Full API intent / Partial browser semantics
USB	Yes	Implemented	Full / Partial
USBDevice	Yes	Implemented	Full / Partial
USBConfiguration	Yes	Implemented	Full / Partial
USBInterface	Yes	Implemented	Full / Partial
USBAlternateInterface	Yes	Implemented	Full / Partial
USBEndpoint	Yes	Implemented	Full / Partial
USBConnectionEvent	Yes	Implemented; exposed on window	Partial
USBInTransferResult	Yes	Implemented	Partial
USBOutTransferResult	Yes	Implemented	Partial
USBIsynchronousInTransferResult	Yes	Implemented with backend limitations	Limited
USBIsynchronousOutTransferResult	Yes	Implemented with backend limitations	Limited
Isochronous packets	Yes	Implemented; packet-level fidelity limited by public pyusb API	Limited

3. USBDevice methods and state

Feature / Operation	Chrome / Chromium	v0.0.5b2	Assessment
open()	Opens authorized device	Implemented through Python bridge	Partial
close()	Closes device	Implemented; JS state is reset correctly	Partial
forget()	Forgets permission for device	Host-managed permission removal	Partial / Host-specific
selectConfiguration()	Selects configuration	Implemented; per-interface JS claim state reset	Partial
claimInterface()	Claims interface	Implemented; redundant claim is a spec-correct no-op	Partial
releaseInterface()	Releases interface	Implemented; unclaimed release is a no-op	Partial
selectAlternateInterface()	Selects alternate interface	Implemented; protected-class alternates rejected	Partial
controlTransferIn()	Control IN transfer	Implemented	Partial
controlTransferOut()	Control OUT transfer	Implemented; endpoint-recipient checks are alt-aware	Partial
clearHalt()	Clears endpoint halt	Implemented	Partial
transferIn()	Bulk/interrupt IN	Implemented	Partial
transferOut()	Bulk/interrupt OUT	Implemented	Partial
isochronousTransferIn()	Isochronous IN	Implemented; real hardware/backend dependent	Limited
isochronousTransferOut()	Isochronous OUT	Implemented; real hardware/backend dependent	Limited
reset()	Resets USB device	Implemented; JS interface state is reset	Partial
opened/configuration state	Browser tracks WebUSB state	Bridge/native backend tracks corresponding state	Partial

4. requestDevice() and filters

Feature / Operation	Chrome / Chromium	v0.0.5b2	Assessment
requestDevice()	User-mediated chooser required	Native chooser / host bridge	Partial
filters	Standard WebUSB filters	Supported and independently validated	Full / Partial
exclusionFilters	Supported by WebUSB model	Supported	Partial
vendorId / productId	Supported	Supported	Full
serialNumber	Supported where exposed	Supported; chooser can disambiguate by serial number	Partial
classCode / subclassCode / protocolCode	Supported	Supported with strict validation	Partial
Filter validation	Browser validates structure	Python + JS-side validation and numeric boundaries	Close / Partial
Chooser cancellation	Promise rejection in browser	Mapped to WebUSB-style NotFoundError where applicable	Partial
Device selection UI ownership	Browser UI	Application/native UI	Different
User activation	Browser gesture requirement	requestDevice() validates user activation	Partial

5. getDevices() and permission semantics

Feature / Operation	Chrome / Chromium	v0.0.5b2	Assessment
getDevices()	Returns devices previously granted to origin	Returns already-granted devices for requesting origin	Partial
Origin scoping	Browser-enforced	Origin-aware host-side handling	Partial
Persistent authorization	Browser permission store	Host-side permission state	Different
Permission revocation	Browser UI / permissions APIs	Host-side permission management	Different
Cross-origin visibility	Blocked by origin permissions	Origin checks intended to prevent leakage	Partial
Frame attribution	Browser knows requesting frame	Frame-aware origin handling; stronger QWebEngineFrame model when available	Partial
Arbitrary system enumeration	Not exposed to page	Not exposed as unrestricted page API	Close
Host pre-authorization	Not a normal page API	grant_device_for_origin() available to trusted host applications	Host-specific
Virtual descriptor replay	Not a browser permission feature	Captured descriptors can be replayed through virtual devices	Host-specific / Extra

6. Secure context and Permissions Policy

Feature / Area	Chrome / Chromium	v0.0.5b2	Assessment
Secure Context	WebUSB requires secure context	QtWebEngine host controls navigation/origin environment; not identical to Chrome policy	Partial / Different
HTTPS origin	Standard browser security boundary	Supported as an origin input	Partial
Permissions Policy: usb	Browser-enforced policy applies	Must be enforced/controlled by QtWebEngine host environment	Partial / Host-dependent
iframe policy inheritance	Browser-defined	Frame-aware handling exists, but not identical to Chromium browser-process policy	Partial
Opaque origins	Browser-defined handling	Host/polyfill behavior can differ	Limited
User activation	Browser activation model	Host/polyfill enforces where applicable	Partial
Browser UI permission prompt	Chrome-controlled	Application/native chooser	Different
Frame-origin isolation	Chromium frame model	QWebEngineFrame-based model when available; older environments fall back to main-frame-only behavior	Partial / Version-dependent

7. Security restrictions and hardening

Feature / Area	Chrome / Chromium	v0.0.5b2	Assessment
Protected USB interface classes	Blocks protected classes	Protected classes rejected; checks also cover transfers, clearHalt, and alternate-interface selection	Close / Partial
Security-key blocklist	Chromium security-key blocklist	Chromium-derived security-key blocklist	Close
Transfer validation	Browser validates transfer parameters	Bridge validates transfer parameters and boundaries	Close / Partial
Transfer-size safety	Chrome has browser-specific limits	32 MiB warning threshold + separate 512 MiB host safety ceiling	Different by design
Host resource limits	Browser process/resource controls	Host-side limits and hardening	Different
Origin authorization	Browser permission system	Origin-aware permission manager	Partial
Direct native access from page	Not possible	Page cannot directly call pyusb/libusb; bridge mediates access	Close
Bridge bypass resistance	Browser process boundary	QWebChannel validation + hardening of host-only management APIs	Partial / Host-dependent
Malicious descriptors	Browser USB stack validates behavior	Native/backend validation and hardening	Partial
Resource exhaustion	Browser safeguards	Host-side safety limits and hardening	Partial
Base64 validation	Browser/Web API semantics	Canonical validation; malformed payloads map to DataError	Close / Partial
Chooser payload boundary	Browser internal	Options JSON >64 KiB rejected	Host-specific safety
Simultaneous-open handles	Browser-managed resources	Per-origin simultaneous-open-handle limits	Host-specific

Protected interface classes

The README states that eight classes are rejected: Audio, HID, Mass Storage, Hub, Smart Card, Video, Audio/Video, and Wireless Controller. The protection is also enforced for transfers, clearHalt(), and selectAlternateInterface(), so claimInterface() cannot simply be bypassed by switching to a protected alternate or addressing its endpoint directly.

8. Transfer behavior

Feature / Area	Chrome / Chromium	v0.0.5b2	Assessment
Control transfers	Supported	Supported	Partial
Bulk transfers	Supported	Supported	Partial
Interrupt transfers	Supported	Supported	Partial
Isochronous transfers	Supported	Supported with backend limitations	Limited
Stall handling	WebUSB result/status mapping	Mapped where possible	Partial
Babble handling	Browser/backend-defined WebUSB result	Mapped where possible	Partial
DataView / ArrayBuffer model	Web API data types	Polyfill maps to compatible JS objects	Partial
32 MiB reference	Important Chromium implementation reference	console.warn() above 32 MiB rather than unconditional rejection	Different
Host ceiling	Browser-internal	512 MiB host-side safety ceiling	Host-specific
Non-uniform isochronous packets	Supported where backend permits	Still limited because public pyusb does not expose complete per-packet lengths	Limited
Control transfer maximum	Browser/backend defined	65,535-byte maximum control transfer length in diagnostics	Host-specific / Different

Virtual USB testing

v0.0.5b2 adds `VirtualUsbDevice.from_descriptor()`. A real device descriptor can be captured in the shape already produced by the bridge, then reconstructed as a virtual device for CI, bug reproduction, or repeatable tests without keeping the physical device attached. This is a host-side testing capability, not a Chrome WebUSB feature.

9. Hotplug and events

Feature / Area	Chrome / Chromium	v0.0.5b2	Assessment
device connect event	Supported	Hotplug monitoring and bridge event path	Partial
device disconnect event	Supported	Hotplug monitoring and bridge event path	Partial
Origin filtering of events	Browser permission/origin model	Origin-aware event delivery intended	Partial
Cross-origin hotplug leakage prevention	Browser-enforced	Explicit hardening/tests target this case	Partial
Native watcher	Browser USB service/backend	pyusb/libusb backend	Different
Timing guarantees	Browser implementation-defined	Backend/OS dependent	Different
First-poll behavior	Already-attached devices do not generate a new connect event	First poll establishes baseline; later diffs generate events	Close / Partial
Device identity limitation	Browser-managed device identity	Hotplug watcher identity remains vendor_id + product_id	Limited

10. Error and exception behavior

Feature / Area	Chrome / Chromium	v0.0.5b2	Assessment
DOMException-style API errors	Standard browser exceptions	Polyfill maps native failures to JS/WebUSB-style errors	Partial
NotFoundError	Missing/cancelled selection cases	Used for chooser cancellation and missing devices where applicable	Partial
SecurityError	Browser security boundary	Host/polyfill security checks	Partial
NetworkError	Browser/backend failures	Bridge/backend failures may be mapped	Partial
InvalidStateError	State-machine errors	Bridge tracks device state	Partial
OperationError	Transfer/backend failures	Backend failures mapped where possible	Partial
Exact message compatibility	Not guaranteed across browser versions	Not intended as byte-for-byte Chrome clone	No
Error timing	Browser-defined	Python/Qt/backend timing can differ	Different

11. DevTools and diagnostics

Feature / Area	Chrome / Chromium	v0.0.5b2	Assessment
DevTools inspection	Chrome DevTools	QtWebEngine DevTools / F12	Close for inspection
Implementation namespace	Chrome internal objects are not public API	window.__pysideWebUSB	Host-specific
Granted-device diagnostics	Browser permission UI/internal state	listGrantedDevices()	Host-specific
Transfer-limit diagnostics	Browser-specific internals	explainTransferLimits()	Host-specific
Bridge diagnostics	Not a WebUSB page API	bridgeInfo()	Host-specific
Environment report	Not a WebUSB page API	environment_report() / format_environment_report()	Host-specific
Doctor CLI	No equivalent public WebUSB API	pyside6-webusb-doctor	Host-specific
JSON diagnostics	No equivalent public WebUSB API	--json / python -m pyside6_webusb --json	Host-specific
QtWebEngine importability	Not a WebUSB API	Diagnostics report QtWebEngineCore/Widgets importability	Host-specific
pyusb backend note	Not a WebUSB API	Diagnostics can report legacy libusb0 fallback information	Host-specific

12. Host-only capabilities

Feature / Area	Chrome / Chromium	v0.0.5b2	Assessment
Host pre-authorization	Not exposed as normal page API	grant_device_for_origin()	Extra
Python bridge	No	Yes	Extra
QWebChannel integration	No	Yes	Extra
pyusb/libusb backend	Browser-internal implementation differs	Yes	Different / Extra
Rust acceleration	Not a WebUSB API	Optional	Extra
Environment diagnostics	Not a WebUSB API	Yes	Extra
Application-controlled chooser	Browser chooser	Host application controls native chooser integration	Different
Embedded deployment	Chrome is a browser	Designed for PySide6 embedded applications	Different
VirtualUsbDevice / backend	Not a browser feature	Repeatable simulated USB devices	Extra
Descriptor replay	Not a browser feature	from_descriptor() for captured device shapes	Extra

13. TypeScript and API surface

Feature / Area	Chrome / Chromium	v0.0.5b2	Assessment
Web IDL/API shape	Defined by WebUSB specification	TypeScript definitions and JS polyfill	Close / Partial
USBDevice typing	Native browser IDL	Provided TypeScript definitions	Close
USBConfiguration typing	Native browser IDL	Provided	Close
USBInterface typing	Native browser IDL	Provided	Close
USBEndpoint typing	Native browser IDL	Provided	Close
USB / USBConnectionEvent on window	Browser-provided	Exposed if not already defined	Close / Partial
Runtime identity semantics	Native browser objects	Polyfill/bridge objects	Different
Prototype identity / instanceof	Browser-native Web IDL objects	v0.0.5b2 adds real EventTarget behavior for navigator.usb	Improved / Partial
Byte-for-byte Chrome behavior	Not applicable as a compatibility guarantee	Explicitly not a goal	No

14. Release validation snapshot

Package: 0.0.5.post6 (informal label v0.0.5b2)

Status: Experimental Beta

Automated Python + security-audit tests: 211 passed, 2 skipped

Additional checks: Node polyfill tests and TypeScript strict checks pass; direct execution of the relevant Python test files was also verified after the v0.0.5b2 fixes.

Important limitation: automated tests do not replace broad real-device testing, especially for isochronous transfers and non-uniform packet lengths.

Key v0.0.5b2 changes reflected in this matrix

- FrameOriginTracker now stops rescanning when its QWebEnginePage is destroyed and supports explicit disconnect().
- Endpoint-recipient control-transfer protected-class checking is now alternate-setting aware, avoiding a false block of a benign currently selected endpoint while preserving the defense-in-depth block for protected candidates.
- VirtualUsbDevice validates malformed version inputs and resolves configuration/interface names correctly.
- VirtualUsbDevice.from_descriptor() adds descriptor capture/replay for repeatable hardware-shaped testing.
- Documentation and package metadata identify the release as v0.0.5b2 / 0.0.5.post6.

15. Overall compatibility assessment

Dimension	Assessment	Reason
API surface	High / Partial	Major WebUSB objects, methods, filters, events, and TypeScript surface are implemented, but the objects remain polyfill/bridge objects.
Basic device selection	High / Partial	Native chooser replaces Chrome browser UI; user activation and cancellation are mapped through the host bridge.
Permission model	Medium-High	Origin-aware and granted-device handling exists, but storage/enforcement is host-side.
Security model	Medium-High	Multiple protections exist, but the browser-process security boundary is different.
Transfer compatibility	High for common transfers	Control/Bulk/Interrupt are implemented; backend behavior can differ.
Isochronous compatibility	Limited	Physical-hardware and packet-length edge cases remain less mature than ordinary transfers.
Chrome implementation equivalence	Low by definition	Architecture is fundamentally different.
WebUSB application portability	Potentially high	Applications using standard API patterns can be compatible, but browser assumptions may fail.
Production readiness	Experimental Beta	v0.0.5b2 remains an experimental beta release.

16. Important compatibility conclusions

1. **pyside6-webusb should be described as WebUSB-compatible, not as a Chrome WebUSB clone.** Its API surface is deliberately modeled after WebUSB, while the implementation is a Qt/Python/native bridge.
2. The strongest compatibility area is the ordinary WebUSB programming model: navigator.usb, device objects, filters, permission concepts, common Control/Bulk/Interrupt transfers, and WebUSB-style errors.
3. The largest differences remain browser security architecture, permission storage, native chooser ownership, Permissions Policy enforcement, exact DOMException behavior, runtime object identity, and backend-specific USB behavior.
4. Isochronous transfers require the most caution. The implementation exists, but public pyusb does not expose enough per-packet information for complete fidelity, and broad real-hardware verification remains an explicit experimental area.
5. The 32 MiB transfer behavior is intentionally not identical to Chrome: pyside6-webusb warns above 32 MiB and applies a separate 512 MiB host-side safety ceiling. This is a deliberate compatibility/security trade-off.
6. Host-only functions such as pre-authorization, diagnostics, virtual USB devices, and descriptor replay are additional capabilities, not WebUSB compatibility features.
7. v0.0.5b2 strengthens lifecycle handling, alternate-setting security logic, virtual-device testing, and release validation while retaining the project's conservative stance toward Chrome equivalence.

Reference basis

This document is an English v0.0.5b2 update of the supplied **PySide6-WebUSB Compatibility Chart v0.0.5a2**. The supplied matrix establishes Chrome/Chromium as the comparison baseline and uses conservative compatibility judgments. The v0.0.5b2 source archive supplies the release README, changelog, and validation notes used to update the matrix. Claims not supported by those materials have not been added as verified behavior.

Document note: compatibility is version-, operating-system-, backend-, and hardware-dependent. This matrix is not a claim of byte-for-byte Chrome behavior.